

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green rays.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

Prüfungsleistung (NEU)

- **Portfolio**

- 5 aus 7 Übungen (10 Punkte pro Übungsblatt)

50

- 5 Teilprojekte

- TRAM (10)

- Lexer+Parser (20)

- Code-Erzeugung (25)

90

- Kontrollflussanalyse (10)

- Datenflussanalyse (25)

- **Abgabe des Portfolio bis zum 17.3.2026**

140



- Operator Präzedenz / Assoziativität

```
precedence = (  
    ('left', 'SEMICOLON'),  
    ('left', 'IN'),  
    ('left', 'ELSE'),  
    ('left', 'COMMA'),  
    ('left', 'ASSIGN'),  
    ('nonassoc', 'RELOP'),  
    ('left', 'PLUS', 'MINUS'),  
    ('left', 'MUL', 'DIV'),  
    ('right', 'UMINUS')  
)
```

- ELSE benötigt höhere Präzedenz als SEMI !

if a==0 then 3 else 4 ; 5

a > b > c

a == b == c

4 - -3

4 - -a + b

4 - -(a + b)

Projekt IV

Aufbau des Kontrollflussgraphen

**Siehe
Aufgabenblatt in
StudIP**

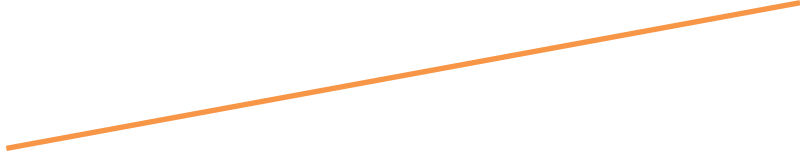
- Interprozedural
 - Funktionsaufruf $g(\dots)$ führt in (Sub-)Graph von $g(\dots)$
 - Vgl. Vorlesung
- Sie können Bibliotheken mit speziellen Datenstrukturen für Graphen verwenden.
- Empfehlung:
 - Implementieren Sie den Graphen selbst.
 - Zuerst als einfach verkettete Datenstruktur, in der Knoten Kinder haben.
 - Wenn der Graph fertig aufgebaut ist, werden zu allen Knoten noch Verweise auf die Eltern hinzugefügt.
 - Die doppelt verkettete Datenstruktur ist zum Entfernen der Zwischenknoten hilfreich.

- Eigene Klassen
 - Knoten:

Verweis auf den Knoten im AST, der mit dem Knoten im CFG zusammenhängt

```
class CFG_Node:
    _counter = 0

    def __init__(self, astnode=None, children=set()):
        self.astnode = astnode
        self.children = children
        self.parents = set() # computed, once graph is built
        CFG_Node._counter += 1
        self.id = CFG_Node._counter
```



- Unterklassen: CFG_Diamond, CFG_START, CFG_END, CFG_CALL und CFG_RET

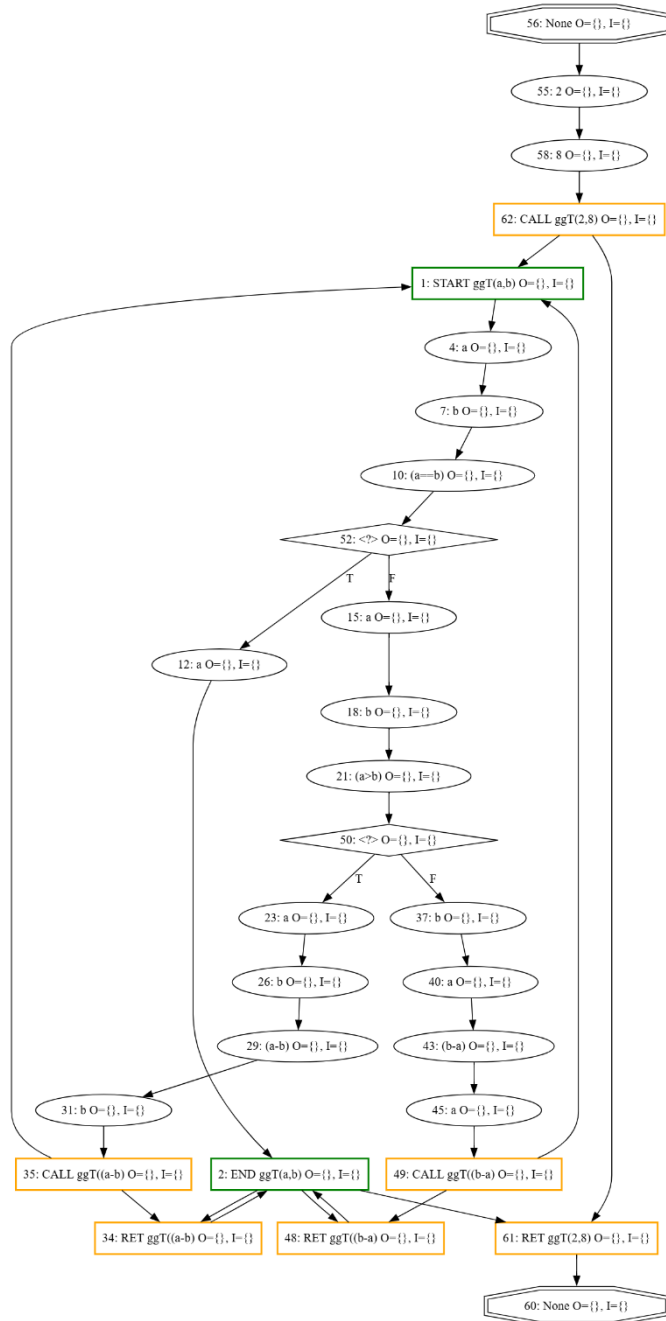
- Eigene Klassen (Interfaces)
 - CFG:

```
class CFG:  
    def __init__(self, inNode, outNode):  
        self.inNode = inNode  
        self.outNode = outNode
```

**Ein- und
Ausgangsknoten
des CFG**

- Berechnung des CFG analog zur Codeerzeugung durch zusätzliche Methode, die für jeden Knotentyp des AST implementiert wird:

```
class VAR(syntax.VAR):  
    def cfa(self, psi):  
        outNode = CFG_Node()  
        node = CFG_Node(self, {outNode})  
        inNode = CFG_Node(None, {node})  
        return CFG(inNode, outNode)
```



```

let ggT(a, b) {
  if (a == b) then
    a
  else if (a > b) then
    ggT(a-b, b)
  else
    ggT(b-a, a)
} in ggT(2, 8) // result should be 2

```

